EECE.4520 Microprocessor System II and Embedded System Design

Theodore Skafidas

02022321

Team T

Date of Completion: Oct 6, 2025

Demonstration Method: in person

Lab 1

Traffic Light Controller

I. DESIGN

This section outlines the hardware and software design principles used to create the traffic light controller.

A. Hardware Design

The design used an ATmega2560 microcontroller to communicate with the physical components of the traffic light. A buzzer along with three LEDs with colors' red, green, and yellow are connected to the PWM output pins of the microcontroller, followed by one limiting resistor each.

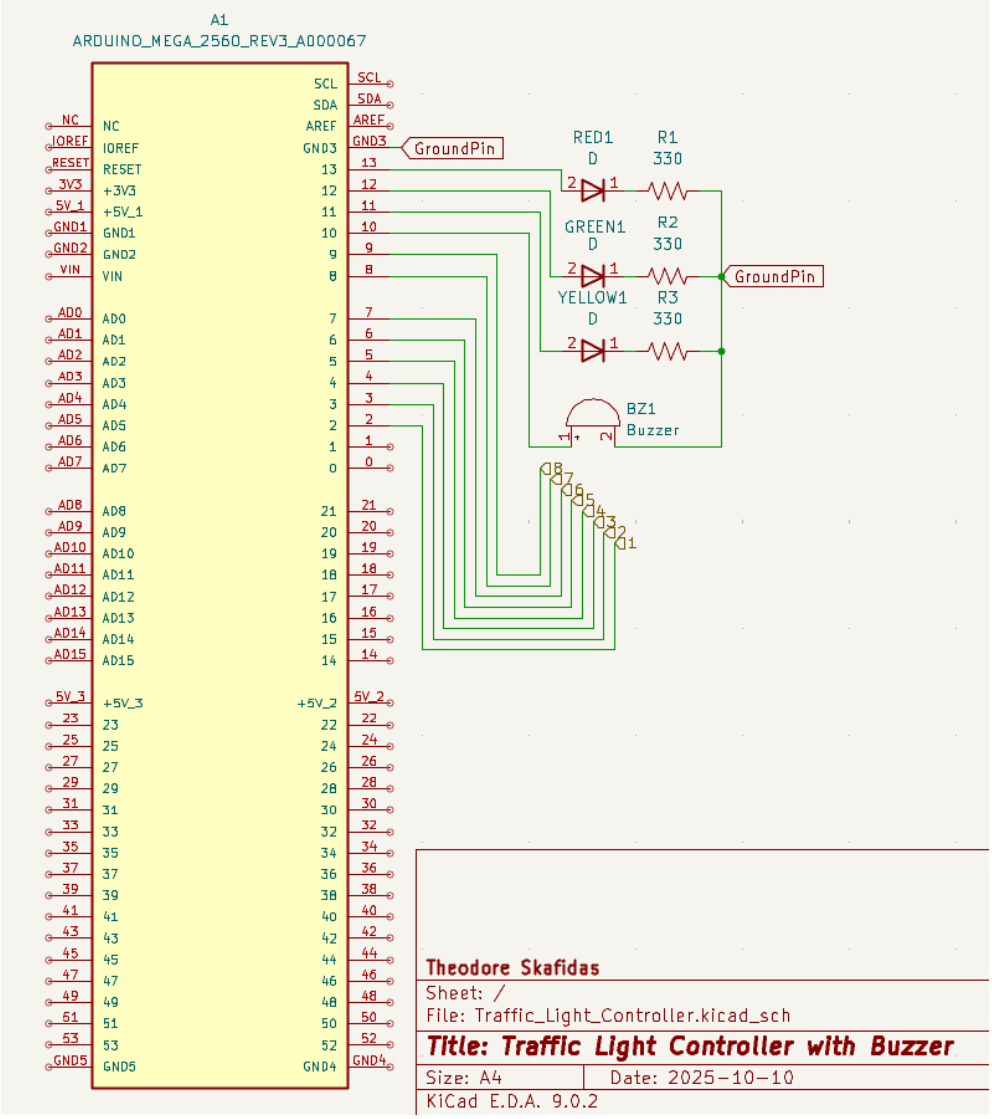

Figure 1 - Traffic Light Controller Schematic

The global connectors [8-1] are meant to correspond to the pins connected to the physical keypad. The figure below, taken from the 4x4 keypad datasheet, shows the numbered pins corresponding to the rows and columns of the keypad. The numbers for these pins are directly translated into the full schematic for a more complete picture.

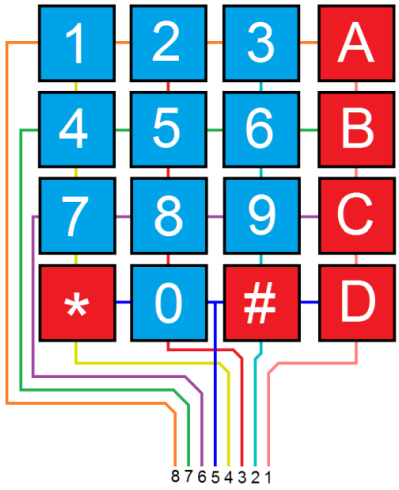

Figure 2

B. Software Design

The following link contains the complete code used in this lab:

https://github.com/Tf3tus/Lab1-Traffic-Light-Controller.git

The software was designed around a finite-state machine (FSM) to manage the distinct modes of operation for the traffic light controller. This event-driven approach allows the system to react to both timed events and user input from a keypad. Starting the software design was simple with the ability to translate the state machine into a switch-case loop. The core `switch` statement inside the main `loop()` evaluates the system's current state on every pass. The states, conditions, and outputs are defined by the mathematical model presented in the report. Transitions between states are triggered by keypad inputs or by the 1 Hz timer interrupt that decrements a global countdown timer.

One key design consideration was the implementation of non-blocking code for all flashing LED sequences. Instead of using the `delay()` function, which could freeze the processor and prevent further input, the software uses the Arduino `millis()` function. This allows the system to manage simultaneous operations: polling the keypad, executing state machine logic, and toggling LEDs at precise intervals.

The software is implemented using both C++ and assembly programming languages. C++ manages high-level program logic, while direct hardware I/O for controlling the LEDs is performed using inline AVR assembly.

Pin assignments were based on the ATmega2560 manual. The necessary pins need to drive digital output with analog input, meaning the PWM digital output pins are best suited for the task.

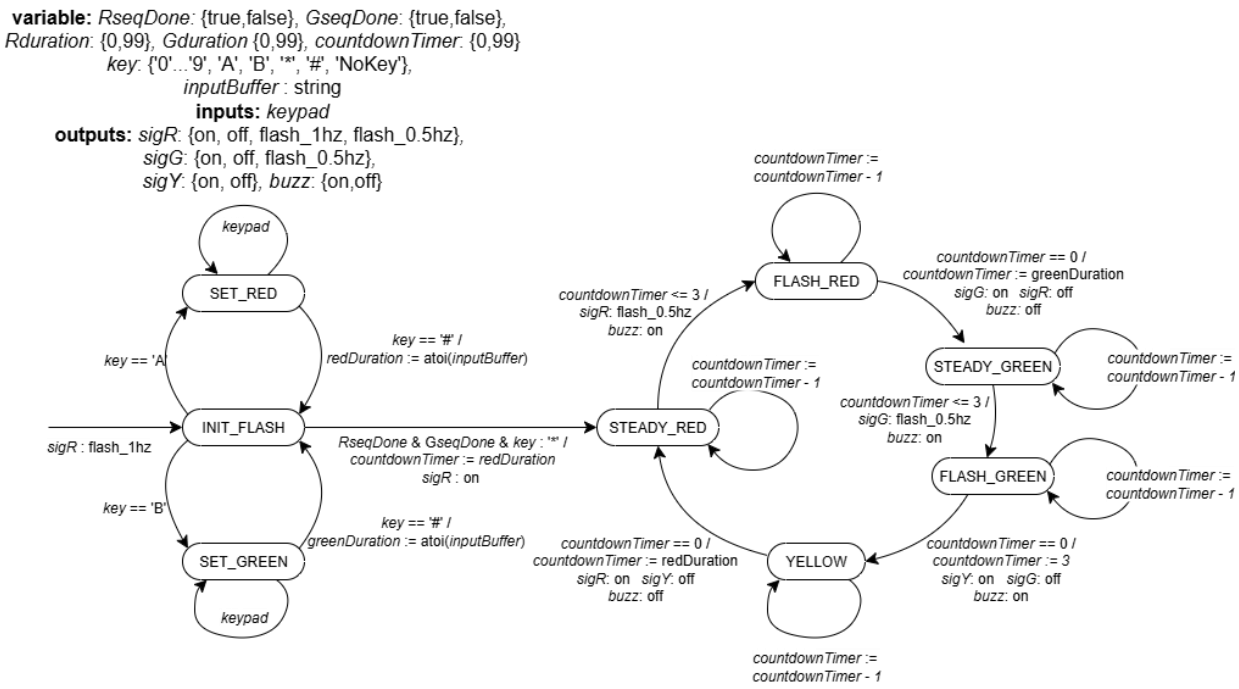

Figure 3 – Finite State Machine Diagram

The design was represented using a mathematical model of the finite-state machine diagram.

States: machine states with description

The following information is a list of possible states with their descriptions.

Q = {INIT_FLASH, SET_RED, SET_GREEN, STEADY_RED, FLASH_RED, STEADY_GREEN, FLASH_GREEN, YELLOW }

INIT_FLASH – red flashes until durations are set and "*" is pressed.

SET_RED – reading keypad entry for red duration.

SET_GREEN - reading keypad entry for green duration.

STEADY_RED- Red ON, count down duration.

FLASH_RED – last 3 seconds before switch, red flashing.

STEADY_GREEN – Green ON, count down duration.

FLASH_GREEN – last 3 seconds before switch, green flashing.

YELLOW – yellow light ON for 3 seconds

Initial State:

q_0 = INIT_FLASH

Final State:

F = null

Conditions: possible conditions used to determine machine state

Σ = {Key_A, A_seq_done, Key_B, B_seq_done, Key_Star, Key_Hash, Timer_le3, Timer_expired}

Key_A = set red duration

A_seq_done = red duration is set and "#" is pressed

Key_B = set green duration

B_seq_done = green duration is set and "#" is pressed

Key_Star = "*" key pressed (to start or confirm)

Key_Hash = "#" key pressed (to finalize the set time of durations)

Timer_le3 = timer still running, 3 seconds or fewer left

Timer_expired = countdown finished, change light

Outputs:

O = {Red_on, Red_flash, Green_on, Green_flash, Yellow_on, Red_flash_slow, Buzzer_on}

Red_on = Red light steadily on

Red_flash = Red light flashing (0.5s ON / 0.5s OFF)

Green_on = Green light steadily on

Green_flash = Green light flashing (0.5s ON / 0.5s OFF)

Yellow_on = Yellow light steadily on

Red_flash_slow = Red flashing during startup (1s ON / 1s OFF)

Buzzer_on = buzzer active (3s before each light change)

Transitions:

NOTE: *Unlisted input conditions for a state means the state remains the same*

$\delta : Q \times \Sigma \rightarrow Q$:

δ(INIT_FLASH, Key_A) = SET_RED / Red_flash_slow

δ(INIT_FLASH, Key_B) = SET_GREEN / Red_flash_slow

δ(INIT_FLASH, Key_Star) =

|STEADY_RED (if Red_duration and Green_duration are set)

δ(SET_RED, A_seq_done) = INIT_FLASH

δ(SET_RED, Key_Hash) =

|INIT_FLASH (if Red_duration is set)

δ(SET_GREEN, B_seq_done) = INIT_FLASH

δ(SET_GREEN, Key_Hash) =

|INIT_FLASH (if Green_duration is set)

δ(STEADY_RED, Timer_le3) = RED_FLASH / Red_flash, Buzzer_on

δ(FLASH_RED, Timer_expired) = STEADY_GREEN / Green_on

δ(STEADY_GREEN, Timer_le3) = FLASH_GREEN / Green_flash, Buzzer_on

δ(FLASH_GREEN, Timer_expired) = YELLOW / Yellow_on

δ(YELLOW, Timer_expired) = STEADY_RED / Red_on

OUT : Q → {Red_on, Red_flash_slow, Red_flash, Green_on, Green_flash, Yellow_on, Buzzer_on}:

OUT(INIT_FLASH) = Red_flash_slow

OUT(SET_RED) = Red_flash_slow

OUT(SET_GREEN) = Red_flash_slow

OUT(STEADY_RED) = Red_on

OUT(STEADY_GREEN) = Green_on

OUT(FLASH_RED) = Red_flash, Buzzer_on

OUT(FLASH_GREEN) = Green_flash, Buzzer_on

OUT(YELLOW) = Yellow_on, Buzzer_on

C. Results

The traffic light controller was successfully implemented and met all functional requirements outlined in the lab. The system operates according to the finite-state machine and responds correctly to keypad inputs and interrupts. The figure below shows the completed hardware setup.

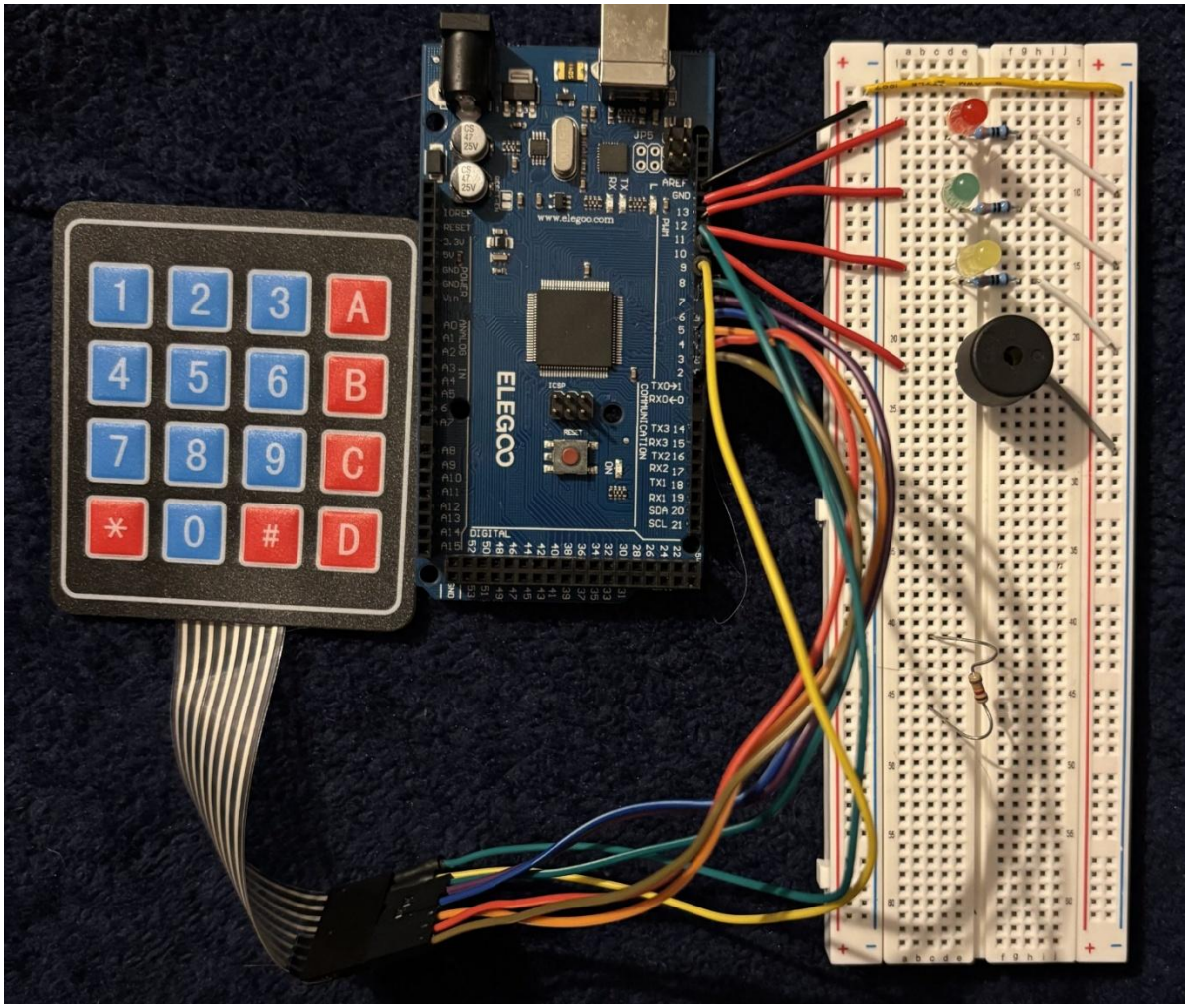

Figure 4 – Physical Traffic Light Controller

The following results are ordered corresponding to the operational states of the controller.

1. ***System Initialization***
   Upon power-on, the system enters the INIT_FLASH state. The red LED flashes at a rate of 1 Hz (one second on/off). The system stays in this state, polling for input from the keypad. The user can successfully set the durations for the red and green lights by pressing the 'A' or 'B' key, followed by two numbers and the '#' key. For example, pressing the sequence 'A' → '2' → '4' → '#' correctly sets the red light duration to 24 seconds. Similarly, pressing the sequence 'B' → '2' → '0' → '#' set the green duration to 20 seconds. The serial monitor output in *Fig.5* demonstrates a succeful test of this input and its parsing function, with the transition between the INIT_FLASH and SET_RED / SET_GREEN states shown as well.

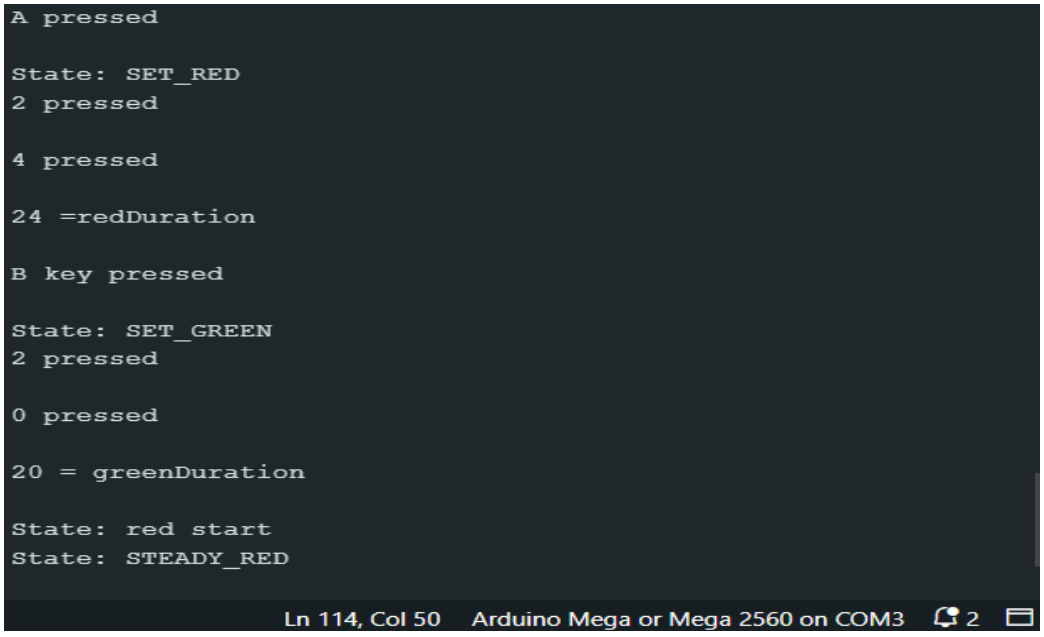

Figure 5 – Serial Monitor Output

2. ***Main Loop***

After both durations are set, pressing the '*' key successfully transitions the system from INIT_FLASH to STEADY_RED, initiatin the main traffic cycle. The controller then cycles through the Red-Green-Yellow pattern as specified:

i. **Red Phase:** The red LED remains solid for the duration specified by the user (*e.g.,* 24 seconds). During the last three seconds, the system transistions to the FLASH_RED state, where the red LED flashes at 2 Hz (0.5 seconds on/off) and the buzzer activates.

ii. **Green Phase:** Once the red timer expires, the system transitions to STEADY_GREEN. The green LED remains solid for the duration specified by the user (*e.g.,* 20 seconds). During the final three seconds, the system transistions to FLASH_GREEN, flashing the green LED at 2 Hz and activating the buzzer.

iii. **Yellow Phase:** Once the green timer expires, the system transitions to the YELLOW state. The yellow LED remains solid and the buzzer is active for a fixed duration of 3 seconds.

The cycle then repeats by returning to the STEADY_RED state and continues until the system is powered off.

II. PROBLEMS

During the development and testing of the traffic light controller, several challenges were encountered. This section documents the key problems and the debugging methods used to resolve them.

A. System Halting After Initial Startup

Problem: The initial software build resulted in the microcontroller flashing an LED a few times then becoming completely unresponsive.

Solution: A debugging process was implemented using serial communication to trace the program's execution with flags. `Serial.println()` statements were inserted at the beginning of each state and logical branches. This allowed us to better locate errors as the program's previous state would be output to the serial monitor. This revealed that the program couldn't switch to the initial state (`INIT_FLASH`) due to an error inside the `setup()` function. The problem was resolved by declaring variables and pinouts *before* the `setup()` function, rather than inside.

B. Inconsistent Keypad Input Registration

Problem: It was perceived that the system was not correctly registering key presses from the 4x4 keypad. When a key was pressed, the serial monitor would either show a different character than the one pressed (*e.g.*, printing '0' for a '#' press) or the code would behave correctly but the debugging output would not match the case.

Solution: The issue was traced to have two possible root causes:

1. **Hardware/Software Mismatch**: The initial suspected problem was a physical swap of two column wires on the breadboard. While the software's character map and column pin assignments matched the physical keypad layout, the wires themselves were swapped to attempt a solution. Upon testing the new wire assembly and referring to the wiring diagram on the 4x4 keypad data sheet, the change was found to have no effect on the faulty serial monitor output.
2. **C++ String Concatenation Behavior:** The misleading debug output was caused by an incorrect attempt to concatenate a `char` and a string literal by using the `+` operator (*e.g.*, `Serial.println(key + " pressed")`). In C++, this results in pointer arithmetic rather than string joining. This means that the command was taking the pointer value of the input key (in this case, the '#' key has a pointer value of 35) instead of the string value (the character '#'). The fix was to use sequential print statements with the correct serial print function (`Serial.print(key); Serial.println(" pressed")`), which called on the input string value instead of the pointer value. This solution provided accurate debugging information.

C. Failure to Store Key Presses

Problem: During testing of the duration-setting states (`SET_RED` and `SET_GREEN`), the system failed to store the key presses for later use while still being able to register the correct key press. For example, after inputting '2' and '4', the `atoi()` function would convert the `inputBuffer` to an incorrect value (*e.g.,* just '4' or no value stored) instead of the expected value '24'.

Solution: A serial print statement involving the contents of the inputBuffer and the value of the inputIndex variable after each press was implemented. This revealed an error in the indexing logic. The code was incrementing the index *before* storing the character in the array (inputIndex++; inputBuffer[inputIndex] = key;). This caused the first digit to be stored in index 1, leaving index 0 uninitialized and corrupting the string entirely. This led to the atoi() function failing as it expected a fully formed string for the input. The problem was solved by incrementing the index *after* storing the character in the array (inputBuffer[inputIndex] = key; inputIndex++;).

D. Failure to Produce Sound in Buzzer

Problem: The buzzer pin would receive a HIGH signal, but no sound was produced.

Solution: Research determined the difference between active and passive buzzers. The component used was a passive buzzer, which requires an oscillating signal (a square wave) to generate sound, whereas the initial code produces only a steady DC voltage via digitalWrite(). The problem was solved by replacing the initial calls with the Arduino tone() and noTone() functions, which generated the necessary frequency to drive the passive buzzer.

E. Unclear Conditions for Duration-Setting Functions

Problem: The red and green duration states allowed additional duration setting after initial duration was set.

Solution: The issue was found in the condition statements of the initial state (INIT_FLASH). The condition statement would only check if the correct button 'A' or 'B' was pressed before moving to one the duration-setting states (if (key == 'A')). This allowed the program to switch to the duration-setting states indefinitely so long as both durations were not set. By utilizing the flags in place that indicated if a duration has been set (RseqDone, GseqDone), the condition statements were modified to check if the boolean value of the respective flag is false (if (key == 'A' && !RseqDone)). This method allowed the duration-setting states to be accessed only once, when the duration isn't set.

F. Incorrect Memory Addressing in Assembly Language I/O Operations

Problem: The final implementation phase involved converting C-based hardware control functions to inline AVR assembly. The initial attempt using sbi (set bit) and cbi (clear bit) instructions resulted in a compile error: Error: operand out of range.

Solution: Reading the AVR instruction set documentation revealed that sbi and cbi are instructions limited to the first 32 I/O registers (addresses 0x00-0x1F). The I/O ports for the selected LED pins on the ATmega2560 were declared as PORTB at 0x25 which is outside the addressable range. Further investigation of the AVR datasheet revealed that the correct I/O address for PORTB is 0x05. Because this address is within the addressable range, the incorrect address (0x25) was replaced with (0x05) and the issue was resolved.

III. TECHNICAL DETAILS

As the sole contributor to this project, I was responsible for all aspects of the design, implementation, and testing of the traffic light controller. The following is a detailed description of my technical contributions:

Hardware Design and Assembly:

- Designed the complete circuit schematic, describing the connections between the ATmega2560, keypad, LEDs, and buzzer.
- Physically assembled and wired the entire circuit on a breadboard, including selecting proper limiting resistors for the LEDs.

Software Architecture and State Machine Design:

- Designed the complete finite-state machine (FSM) that serves as the core of the controller logic. This included, defining all states, the conditions for transitions, and the specific actions to be taken in each state.
- Developed a mathematical model to represent the FSM.
- Designed the software to be non-blocking by using `millis()` function for all flashing light sequences.

C++ Software Implementation:

- Wrote the entire C++ application, using the standard Arduino structure functions and implementing the FSM using a `switch-case` structure.
- Configured the ATmega2560 hardware to generate a 1Hz pulse to interrupt all time-based events and wrote the Interrupt Service Routine (ISR) to manage the countdown timer.
- Implemented logic for parsing user input from the keypad, which included managing the character buffer, tracking an index, and using the `atoi()` function to convert the final string into an integer value for the light durations.

Mixed-Language Programming:

- Replaced the C++ functions for all LED control (`RedLight`, `Yellow`, `GreenLight`) with inline AVR assembly.
- Wrote assembly code to handle direct I/O manipulation, setting and clearing specific bits in the `PORTB` register to control the LEDs.

Debugging and Problem Solving:

- Debugged the entire system from end-to-end. This included using the serial monitor to trace program flaws.
- Found and resolved multiple integration issues between the hardware and software, namely the keypad.
- Solved assembly language compilation errors.

IV. CONCLUSION

This lab provided experience in the design, implementation, and debugging of an embedded system. Key knowledge and skills were gained in many areas. A primary lesson was the design and implementation of a non-blocking finite-state machine (FSM) in C++, which proved to be scalable architecture for managing event-driven systems. The important use of non-blocking code over blocking code ensured the successful execution of interrupts. The project provided hands-on experience in mixed-language programming. It was important to know how to partition the system effectively by using C++ for high-level programming and AVR assembly for low-level I/O interface. The process of debugging assembly errors gave insight into the microcontroller architecture, including I/O register memory mapping. Finally, the project reinforced the routine method of debugging, such as serial tracing, to pinpoint and solve software integration problems.

V. REFERENCES

[1] *4x4 Matrix Membrane Keypad (#27899)*, Parallax Inc, 2011.

[2] *AVR® Instruction Set Manual*, Microchip Technology Inc, 2021.

[3] "Arduino mega2560 re v3." Snapeda. https://www.snapeda.com/parts/ATmega2560

[4] *Elegoo mega 2560 R3*, ELEGOO, 2025.

[5] *Lab 1: Traffic Light Controller*, Luo.Y, 2025